

Metodologia Experimental

Benchmark do Impacto de Mecanismos Criptográficos em Bancos de Dados NoSQL

Caio Sena Freitas e Pedro Henrique T. Pinto — CEFET/RJ, Nova Friburgo

2026

1. Visão Geral

Este documento descreve a metodologia experimental utilizada para avaliar o impacto de mecanismos criptográficos no desempenho de três bancos de dados NoSQL: **MongoDB 7.0**, **Redis** e **Apache Cassandra 4.1**.

O objetivo central é quantificar o custo computacional (throughput, latência, uso de CPU e RAM) introduzido por operações criptográficas integradas ao ciclo de vida de dados (INSERT, READ, UPDATE, DELETE).

2. Estratégia Experimental

A metodologia baseou-se em duas vertentes complementares:

- Carga base com YCSB:** utilizou-se o Yahoo! Cloud Serving Benchmark (YCSB) v0.17.0, com 100.000 registros, para estabelecer o baseline de desempenho dos bancos sem proteção criptográfica, sob os workloads padrão A, B, C e D.
- Benchmark criptográfico customizado:** um conjunto de 9 operações padronizadas, executadas de forma idêntica nos três bancos, combinando algoritmos criptográficos com operações reais de persistência.

3. Operações Benchmark Padronizadas

ID	Operação	Descrição
O1	insert_aes256	INSERT com campo criptografado em AES-256-CFB
O2	insert_sha256	INSERT com hash SHA-256 do payload
O3	insert_hmac_sha256	INSERT com HMAC-SHA-256 (chave secreta)
O4	insert_bcrypt	INSERT com hash Bcrypt

		(rounds=10)
O5	insert_argon2	INSERT com hash Argon2id (time_cost=1, memory_cost=65536, parallelism=2)
O6	read_decrypt_aes256	READ + decifração AES- 256-CFB
O7	read_verify_sha256	READ + verificação de integridade SHA-256
O8	update_reencrypt_aes256	UPDATE com recriptografia AES-256-CFB
O9	delete_encrypted_record	DELETE de registro criptografado

Cada operação foi executada **10.000 vezes** por tamanho de payload, para os tamanhos **64, 256, 1.024 e 4.096 bytes**, totalizando 40.000 iterações por operação por banco.

4. Algoritmos Criptográficos

- **AES-256-CFB**: cifra de bloco simétrica em modo Cipher Feedback, adequada para payloads de tamanho variável.
- **SHA-256 / HMAC-SHA-256**: funções de hash e autenticação de mensagens da família SHA-2.
- **Bcrypt** (rounds=10) e **Argon2id** (time_cost=1, memory_cost=64 MB, parallelism=2): funções de derivação de chave para senhas, intencionalmente custosas em CPU/memória.

5. Protocolo de Medição

- Os payloads foram gerados com bytes aleatórios criptograficamente seguros (os.urandom), eliminando vieses de compressão.
- O monitoramento de CPU e RAM foi realizado em uma thread separada, com amostras coletadas a **cada 50 ms** durante a execução de cada teste, via psutil.
- O tempo de execução foi medido com time.perf_counter().
- Para mitigar efeitos de warm-up de JIT e cache do sistema operacional, os testes foram executados na ordem: **Redis → MongoDB → Cassandra**.

6. Métricas Coletadas

Para cada operação e tamanho de payload, foram coletadas:

- **Throughput** (ops/s)
- **Latência média** (ms/op)
- **Uso médio de CPU** (%)
- **Uso de pico de CPU** (%)
- **Uso médio de RAM** (%)
- **Uso de pico de RAM** (%)

7. Dataset

Foi utilizado o dataset **RT-IoT2022** (UCI Machine Learning Repository, ID 942), composto por amostras de tráfego de rede de dispositivos IoT, para a etapa de **importação inicial / massa de dados realista**. Os testes criptográficos em si utilizaram payloads sintéticos gerados via `os.urandom`, garantindo controle total sobre o tamanho dos dados testados.

8. Isolamento de Hardware

Para eliminar a interferência de I/O compartilhado entre os bancos, **cada SGBD foi instalado em um disco físico (HD) exclusivo**, na mesma máquina, sob o mesmo sistema operacional (Debian GNU/Linux 12). Detalhes completos do ambiente estão disponíveis no documento **Hardware**.

9. Limitações

- Implantação **single-node** (sem cluster), o que não reflete o comportamento de Cassandra em ambientes distribuídos para os quais foi projetado.
- Execução **serial**, sem concorrência de múltiplas threads de cliente nos benchmarks criptográficos.
- Redis configurado **sem persistência em disco** (`save ""`), favorecendo seu throughput em relação a configurações de produção que exigem durabilidade.